MAHATMA GANDHI
INSTITUTE OF TECHNOLOGY (Autonomous)
Kokapet(Village), Gandipet, Hyderabad, Telangana - 500075. www.mgit.ac.in
Affiliated to
Re-accredited by
MOTIVATE INNOVATE EMPOWER 27 YEARS

Grouping Anagrams Using Strings and Sorting

Presented by : P Suryatej Kaasyap (25261A04H6)
Y Lohith (25261A04K1)

Academic Year : 2025-26

Project Guide : Dr. Y. Praveen Kumar Reddy

Date :15-05-26

Abstract

- ✓ Anagrams are words formed by rearranging the letters of another word while preserving all original characters exactly once. Identifying and grouping anagrams is a common problem in string processing and algorithm design.

- ✓ This project focuses on designing and implementing an efficient program to group a collection of words into corresponding anagram sets. The implementation is carried out using the C programming language with concepts such as arrays, strings, sorting techniques, and comparison operations.

- ✓ The core idea behind the solution is to convert every word into a canonical form by sorting its characters alphabetically. Words sharing the same sorted representation are grouped together as anagrams.

- ✓ The project demonstrates efficient data organization and string manipulation techniques while providing practical exposure to algorithmic problem solving.

Introduction

- ✓ String processing plays an important role in computer science applications such as text analysis, search optimization, spell checking, and natural language processing. One interesting string manipulation problem is the identification of anagrams.

- ✓ An anagram is formed when the letters of a word are rearranged to produce another valid word while maintaining the same character frequency. For example, the words listen and silent are anagrams.

Anagram Algorithm

Algorithm :

1. **Start**

2. **Input** the number of words `n`.

3. **Read** all words into the array words[ ][ ].

4. For each word:

   ✓ Copy the word into another array sorted[ ][ ].

   ✓ Sort the characters of the copied word alphabetically using sortString().

5. Initialize a visited[ ] array with all values as 0.

6. For each word i from 0 to n-1:

   - ✓ If visited[i] is 1, skip it.
   - ✓ Print the current word.
   - ✓ Mark visited[i] = 1.

7. Compare the sorted form of the current word with all remaining words:

   - ✓ If sorted[i] equals sorted[j] using strcmp( ):
   - ✓ Print the word.
   - ✓ Mark visited[j] = 1.

8. Print a new line after each anagram group.
9. Stop.

Time Complexity

For grouping and processing words, time required:

$$\boldsymbol{O(n \cdot k \log k)}$$

where:

✓ n = number of words

✓ k = maximum word length

Time Usage Breakdown:

✓ Sorting each word → $O(k \log k)$

✓Sorting all words → $O(n \cdot k \log k)$

✓Comparing sorted words (grouping) → $O(n^2 \cdot k)$

Space complexity

For storing and processing words, memory requires:

$$O(n \cdot k)$$

where:

✓ n = number of words

✓ k = maximum word length

Memory Usage Breakdown:

✓ words[MAX][50] → stores input words → O(n · k)

✓ sorted[MAX][50] → stores sorted words → O(n · k)

✓ visited[MAX] → tracking array → O(n)

Example

ANAGRAM GROUPING – EXAMPLE

1. INPUT

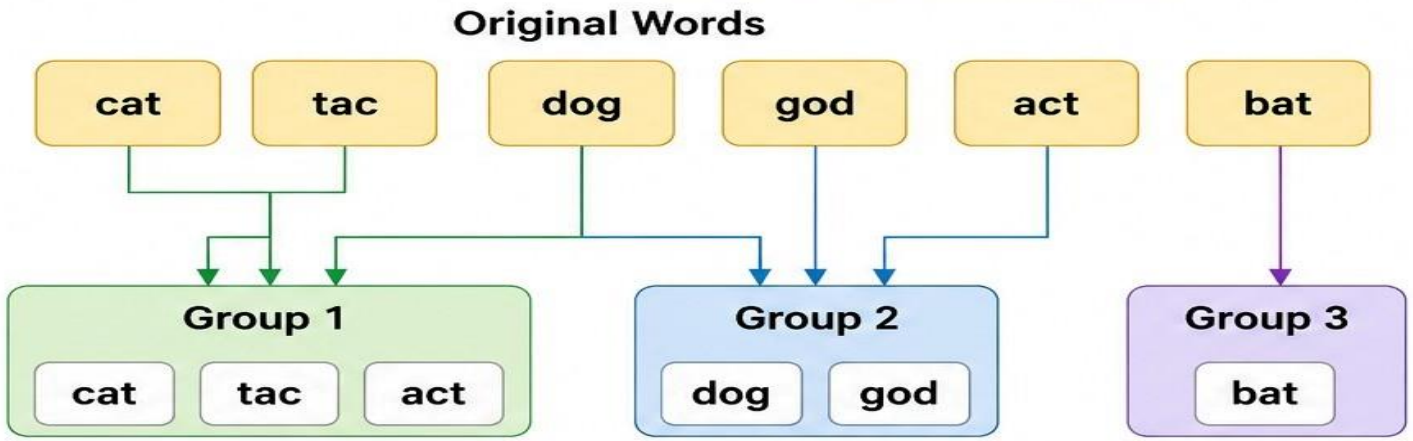
- Number of words (n) = 6
- Words:

cat | tac | dog | god | act | bat

2. SORT EACH WORD

Original Word	Sorted Word
cat	act
tac	act
dog	dgo
god	dgo
act	act
bat	abt

3. GROUP ANAGRAMS



ANAGRAM GROUPS (OUTPUT)

Group 1: cat tac act

Group 2: dog god

Group 3: bat

Explanation

Words are grouped together if their sorted form is the same.

Technical Applications

1. Compiler Design

   Used in lexical analysis and pattern matching of strings.

2. Database Management Systems (DBMS)

   Helps in grouping similar text records and removing duplicate patterns.

3. Search Engines

   Improves search suggestions by detecting rearranged keywords.

Example:

"silent" ↔ "listen“

4. Cybersecurity & Cryptography

Used in basic encryption/decryption techniques involving character rearrangement.

5. Natural Language Processing (NLP)

Useful in text mining, word similarity checking, and language processing systems.

6. Artificial Intelligence

Used in intelligent word prediction and language-based AI systems.

Full Code Explanation with `"decimal"` and `"claimed"`

Step 1 — Input is Stored

```
words[0] = "decimal"
words[1] = "claimed"
```

Step 2 — `strlen` on each word

```
d  e  c  i  m  a  l  \0
0  1  2  3  4  5  6   7

strlen("decimal") = 7
```

```
c  l  a  i  m  e  d  \0
0  1  2  3  4  5  6   7

strlen("claimed") = 7
```

- Both have **7 characters**
- Loop will run from index **0 to 6**

Step 3 — `strcpy` copies to `sorted[]`

```
sorted[0] = "decimal"    (copy of words[0])
sorted[1] = "claimed"    (copy of words[1])
```

- Original words stay **untouched**
- Only `sorted[]` copies will be modified

Step 4 — `sortString()` sorts each copy

Sorting `"decimal"`:

```
Original:  d  e  c  i  m  a  l
           0  1  2  3  4  5  6

Step by step swapping:
d > c → swap → c e d i m a l
e > a → swap → c a d i m e l
...
Final:     a  c  d  e  i  l  m
```

Sorting `"claimed"`:

```
Original:  c  l  a  i  m  e  d
           0  1  2  3  4  5  6

Step by step swapping:
c > a → swap → a l c i m e d
l > c → swap → a c l i m e d
...
Final:     a  c  d  e  i  l  m
```

Step 5 — `temp` during swapping

Taking `d` and `c` from `"decimal"`:

```
c

str[0] = 'd',  str[2] = 'c'

temp   = 'd'          // save 'd'
str[0] = 'c'          // put 'c' in position 0
str[2] = 'd'          // put saved 'd' in position 2

Result: c...d...  ✓
```

Step 6 — `strcmp` compares sorted versions

```
sorted[0] = "acdeilm"   ← sorted "decimal"
sorted[1] = "acdeilm"   ← sorted "claimed"

strcmp("acdeilm", "acdeilm") = 0   ✓ MATCH!
```

- Both produce **identical sorted strings**
- So they are confirmed **anagrams**

Step 7 — `visited[]` prevents duplicates

```
visited[0] = 0  → not yet printed
visited[1] = 0  → not yet printed
```

- `i = 0` → `visited[0]` is 0 → print `"decimal"` → mark `visited[0] = 1`
- Inner loop finds `sorted[1]` matches → print `"claimed"` → mark `visited[1] = 1`
- Next iteration `i = 1` → `visited[1]` is 1 → skip

Final Output

```
Anagram Groups:
decimal claimed
```

References

- ✓ AI-based tools such as ChatGPT and Gemini used for conceptual understanding, report structuring, and code development support and images.
- ✓ Yashavant P. Kanetkar, Let Us C, BPB Publications.
- ✓ Aho, A. V., Sethi, R., and Ullman, J. D., Compilers: Principles, Techniques, and Tools, Addison-Wesle

Thank You